

EXPERIMENT – 7

Title

To Understand, Use and Work with SQL Joins

Aim / Objective

To understand and implement different types of SQL joins such as **INNER JOIN, LEFT OUTER JOIN, RIGHT OUTER JOIN, FULL OUTER JOIN, SELF JOIN, and CROSS JOIN** for retrieving data from multiple related tables.

Theory

A **JOIN** is an SQL operation used to combine rows from two or more tables based on a related column between them. Joins eliminate data redundancy and allow users to retrieve meaningful information from related tables.

SQL joins are one of the most important concepts in Relational Database Management Systems (RDBMS).

The commonly used joins are:

- INNER JOIN
- LEFT OUTER JOIN
- RIGHT OUTER JOIN
- FULL OUTER JOIN
- SELF JOIN
- CROSS JOIN

Types of Joins

1. INNER JOIN

Returns only the matching records from both tables.

2. LEFT OUTER JOIN

Returns all records from the left table and matching records from the right table. If there is no match, NULL values are returned.

3. RIGHT OUTER JOIN

Returns all records from the right table and matching records from the left table.

4. FULL OUTER JOIN

Returns all records from both tables, including matching and non-matching rows.

5. SELF JOIN

Joins a table with itself.

6. CROSS JOIN

Returns the Cartesian product of two tables.

Learning Objectives

After completing this experiment, students will be able to:

- Understand table relationships.
- Retrieve data using joins.
- Differentiate between various types of joins.
- Apply joins in real-world database applications.
- Combine records from multiple tables efficiently.

Software Requirements

- Oracle Database
- Oracle SQL Developer / SQL*Plus

Algorithm / Procedure

1. Open Oracle SQL Developer.
2. Connect to the Oracle database.
3. Create Department and Employee tables.

4. Insert sample records.
5. Execute different JOIN queries.
6. Observe the output.
7. Stop.

Program

Step 1 : Create Department Table

```
CREATE TABLE Department
(
DeptID NUMBER PRIMARY KEY,
DeptName VARCHAR2(30)
);
```

Step 2 : Create Employee Table

```
CREATE TABLE Employee
(
EmpID NUMBER PRIMARY KEY,
EmpName VARCHAR2(30),
Salary NUMBER,
DeptID NUMBER,
FOREIGN KEY(DeptID)
REFERENCES Department(DeptID)
);
```

Step 3 : Insert Records into Department

```
INSERT INTO Department VALUES(101,'IT');
INSERT INTO Department VALUES(102,'HR');
INSERT INTO Department VALUES(103,'Finance');
INSERT INTO Department VALUES(104,'Marketing');

COMMIT;
```

Step 4 : Insert Records into Employee

```
INSERT INTO Employee VALUES(1,'Rahul',35000,101);
INSERT INTO Employee VALUES(2,'Priya',42000,102);
INSERT INTO Employee VALUES(3,'Aman',39000,101);
INSERT INTO Employee VALUES(4,'Sneha',45000,NULL);

COMMIT;
```

Program 1 : INNER JOIN

```
SELECT  
E.EmpID,  
E.EmpName,  
D.DeptName,  
E.Salary  
  
FROM Employee E  
  
INNER JOIN Department D  
  
ON E.DeptID=D.DeptID;
```

Expected Output

EmpID	EmpName	DeptName	Salary
1	Rahul	IT	35000
2	Priya	HR	42000
3	Aman	IT	39000

Program 2 : LEFT OUTER JOIN

```
SELECT  
E.EmpID,  
E.EmpName,  
D.DeptName  
  
FROM Employee E  
  
LEFT OUTER JOIN Department D  
  
ON E.DeptID=D.DeptID;
```

Expected Output

EmpName	DeptName
Rahul	IT
Priya	HR
Aman	IT
Sneha	NULL

Program 3 : RIGHT OUTER JOIN

```
SELECT  
E.EmpName,  
D.DeptName  
  
FROM Employee E  
  
RIGHT OUTER JOIN Department D  
  
ON E.DeptID=D.DeptID;
```

Expected Output

EmpName DeptName

Rahul	IT
Aman	IT
Priya	HR
NULL	Finance
NULL	Marketing

Program 4 : FULL OUTER JOIN

```
SELECT  
E.EmpName,  
D.DeptName  
  
FROM Employee E  
  
FULL OUTER JOIN Department D  
  
ON E.DeptID=D.DeptID;
```

Expected Output

EmpName DeptName

Rahul	IT
Aman	IT
Priya	HR
Sneha	NULL

EmpName DeptName

NULL Finance

NULL Marketing

Program 5 : SELF JOIN

Create Employee Table with ManagerID

```
ALTER TABLE Employee
```

```
ADD ManagerID NUMBER;
```

Update Manager IDs

```
UPDATE Employee  
SET ManagerID=1  
WHERE EmpID=2;
```

```
UPDATE Employee  
SET ManagerID=1  
WHERE EmpID=3;
```

```
COMMIT;
```

SELF JOIN Query

```
SELECT
```

```
E.EmpName AS Employee,
```

```
M.EmpName AS Manager
```

```
FROM Employee E
```

```
LEFT JOIN Employee M
```

```
ON E.ManagerID=M.EmpID;
```

Expected Output

Employee Manager

Rahul NULL

Priya Rahul

Employee Manager

Aman	Rahul
Sneha	NULL

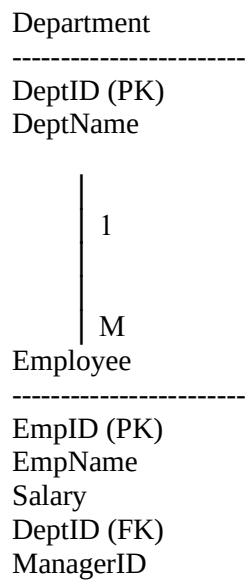
Program 6 : CROSS JOIN

```
SELECT  
  
E.EmpName,  
  
D.DeptName  
  
FROM Employee E  
  
CROSS JOIN Department D;
```

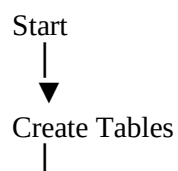
Expected Output

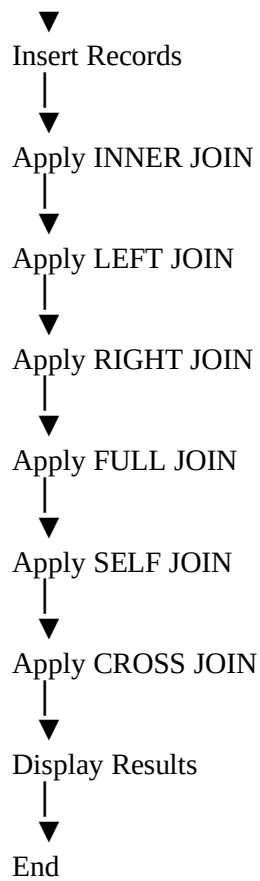
Every employee is paired with every department (Cartesian Product).

Entity Relationship Diagram



Flowchart





Explanation

INNER JOIN

Returns only matching rows from both tables.

LEFT OUTER JOIN

Returns all rows from the left table and matching rows from the right table.

RIGHT OUTER JOIN

Returns all rows from the right table and matching rows from the left table.

FULL OUTER JOIN

Returns all matching and non-matching rows from both tables.

SELF JOIN

Joins a table with itself to retrieve hierarchical data.

CROSS JOIN

Returns every possible combination of rows from both tables.

Observation

- INNER JOIN displayed only matching employee and department records.
- LEFT JOIN displayed all employee records, including those without departments.
- RIGHT JOIN displayed all departments, including those without employees.
- FULL OUTER JOIN combined all matching and non-matching records.
- SELF JOIN successfully displayed employee-manager relationships.
- CROSS JOIN produced the Cartesian product of both tables.

Result

Different SQL joins including **INNER JOIN, LEFT OUTER JOIN, RIGHT OUTER JOIN, FULL OUTER JOIN, SELF JOIN, and CROSS JOIN** were successfully implemented to retrieve related data from multiple tables.

Applications

- Employee and Department Management
- Banking Systems
- Hospital Management Systems
- Student Information Systems
- Library Management Systems
- Inventory Management Systems
- E-Commerce Applications

Advantages

- Retrieves related data efficiently.
- Eliminates data redundancy.
- Supports relational database design.
- Simplifies report generation.
- Improves query flexibility.

Viva Questions

1. What is a JOIN in SQL?
2. Why are joins used?
3. What is the difference between INNER JOIN and OUTER JOIN?
4. What is a LEFT OUTER JOIN?
5. What is a RIGHT OUTER JOIN?
6. What is a FULL OUTER JOIN?
7. What is a SELF JOIN?
8. What is a CROSS JOIN?
9. What is a Cartesian Product?
10. What is the purpose of a FOREIGN KEY in joins?

Lab Assignment

1. Create **Student** and **Course** tables and perform an INNER JOIN.
2. Display all students, including those not enrolled in any course, using a LEFT JOIN.
3. Display all courses, including those without students, using a RIGHT JOIN.
4. Perform a FULL OUTER JOIN between Employee and Department tables.
5. Create an Employee table with a ManagerID column and implement a SELF JOIN.
6. Demonstrate a CROSS JOIN using two sample tables.
7. Compare INNER JOIN and OUTER JOIN with examples.
8. Explain the difference between LEFT JOIN and RIGHT JOIN.

9. Write SQL queries using at least four different types of joins.
10. Explain real-world applications of SQL joins.

Precautions

- Ensure that related tables are created before applying joins.
- Define **PRIMARY KEY** and **FOREIGN KEY** relationships correctly.
- Use appropriate join conditions in the ON clause to avoid incorrect results.
- Be cautious while using **CROSS JOIN**, as it generates a large number of rows.
- Verify the output after each JOIN query.

Learning Outcome

After completing this experiment, students will be able to:

- Understand the concept of table relationships in relational databases.
- Retrieve related information using different types of SQL joins.
- Distinguish between INNER, LEFT, RIGHT, FULL, SELF, and CROSS joins.
- Apply joins to solve real-world database queries.
- Design efficient SQL queries involving multiple related tables.